

RL Project Phase 3 – Final Report

Intelligent Lunar Landing: Deep Reinforcement Learning with Resource Management

Team Members

- Sultan Bahomaid
- Faris Samara
- Yousef Mahmoud
- Feras Nassar

Abstract

We study value-based Deep Reinforcement Learning for the Gymnasium LunarLander-v3 discrete control task under a resource-tracking setting. The environment is modeled as a Markov Decision Process (MDP) with a continuous state and discrete thruster actions. We extend the observation by appending a normalized fuel feature ($\text{fuel_fraction} \in [0,1]$) to track the remaining resource during flight. We implement a Baseline Deep Q-Network (DQN) agent and an Advanced Double DQN (DDQN) agent using identical network architecture and hyperparameters for a fair comparison. Raw training curves are smoothed using a moving average (window = 100 episodes) and all figures are generated in Matplotlib. After training is frozen, each agent is evaluated using a greedy policy over 100 episodes and we report mean reward and stability (standard deviation).

In our run, the Baseline DQN achieves a higher and more stable final smoothed reward (≈ 265 at episode 4000, MA100), while DDQN shows larger mid-training fluctuations and ends lower (≈ 215 , MA100). DQN reaches a higher final success rate ($\approx 89\%$ MA100) compared to DDQN ($\approx 65\%$ MA100).

1. Introduction

The LunarLander task requires controlling a spacecraft to land safely on a designated pad while avoiding crashes. This is challenging because the dynamics are continuous, errors compound over time, and effective control must balance stabilization, descent rate, and lateral drift. We formulate the task as an MDP (S, A, R, γ) where the agent observes a state s , chooses an action a , receives reward r , and transitions to the next state s' . The project goal is to learn a policy that consistently lands safely while tracking fuel as a resource feature in the state.

Phase 3 compares Baseline DQN and Advanced DDQN using the required criteria: (i) learning speed (smoothed reward vs episodes), (ii) loss convergence (smoothed Huber loss vs episodes), (iii) final performance (mean reward over 100 evaluation episodes after training is frozen), and (iv) stability (variance / standard deviation of evaluation reward).

2. Background

Reinforcement Learning (RL) learns a policy that maximizes expected discounted return. Value-based methods learn an action-value function $Q(s,a)$ that estimates the expected return from taking action a in state s .

Deep Q-Networks (DQN) approximate $Q(s,a)$ using a neural network and stabilize training with experience replay and a target network. Double DQN (DDQN) reduces Q-value overestimation by decoupling action selection (online network) from action evaluation (target network).

3. Related Work (Optional)

We implement established algorithms (DQN and DDQN) following the original literature and apply them to Gymnasium LunarLander. Our focus is correct implementation, reproducible evaluation, and required visual comparisons.

4. Methodology

4.1 Environment and State Representation

We use Gymnasium's LunarLander-v3 (Box2D) discrete environment. The original observation is 8-dimensional. We extend it to 9 dimensions by appending $\text{fuel_fraction} \in [0,1]$. Fuel is consumed when the agent fires engines.

4.2 Action Space

The action space is Discrete(4): 0 (do nothing), 1 (fire left engine), 2 (fire main engine), and 3 (fire right engine).

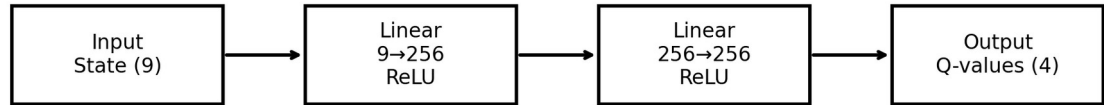
4.3 Reward Shaping Logic

Training uses the native environment reward. Fuel is provided as an additional observation feature. This ensures a clean baseline/advanced comparison.

4.4 Network Architecture

Both agents use the same MLP Q-network: Linear($9 \rightarrow 256$) + ReLU, Linear($256 \rightarrow 256$) + ReLU, Linear($256 \rightarrow 4$).

Figure 1: Q-network architecture (shared by DQN and DDQN).



4.5 Algorithm Implementation Details

Both agents use an experience replay buffer and a periodically updated target network. Training uses ϵ -greedy exploration with decay, Huber loss for TD error, and gradient clipping. The difference is the TD target: DQN uses $\max_a Q_{\text{target}}(s', a)$, while DDQN selects argmax using Q_{online} and evaluates it with Q_{target} .

5. Results & Discussion

5.1 Evaluation Environment (Reproducibility)

Environment: LunarLander-v3 (Gymnasium Box2D). Observation: 9D (8 original + fuel_fraction). Action space: Discrete(4). Plots: Matplotlib; smoothing with MA100. Evaluation: We report end-of-training performance using the final MA100 values and training success-rate curves. (If required by the instructor, a separate 100-episode greedy evaluation can be run after training to report $\text{mean} \pm \text{std}$.)

Hyperparameters (used in both agents)

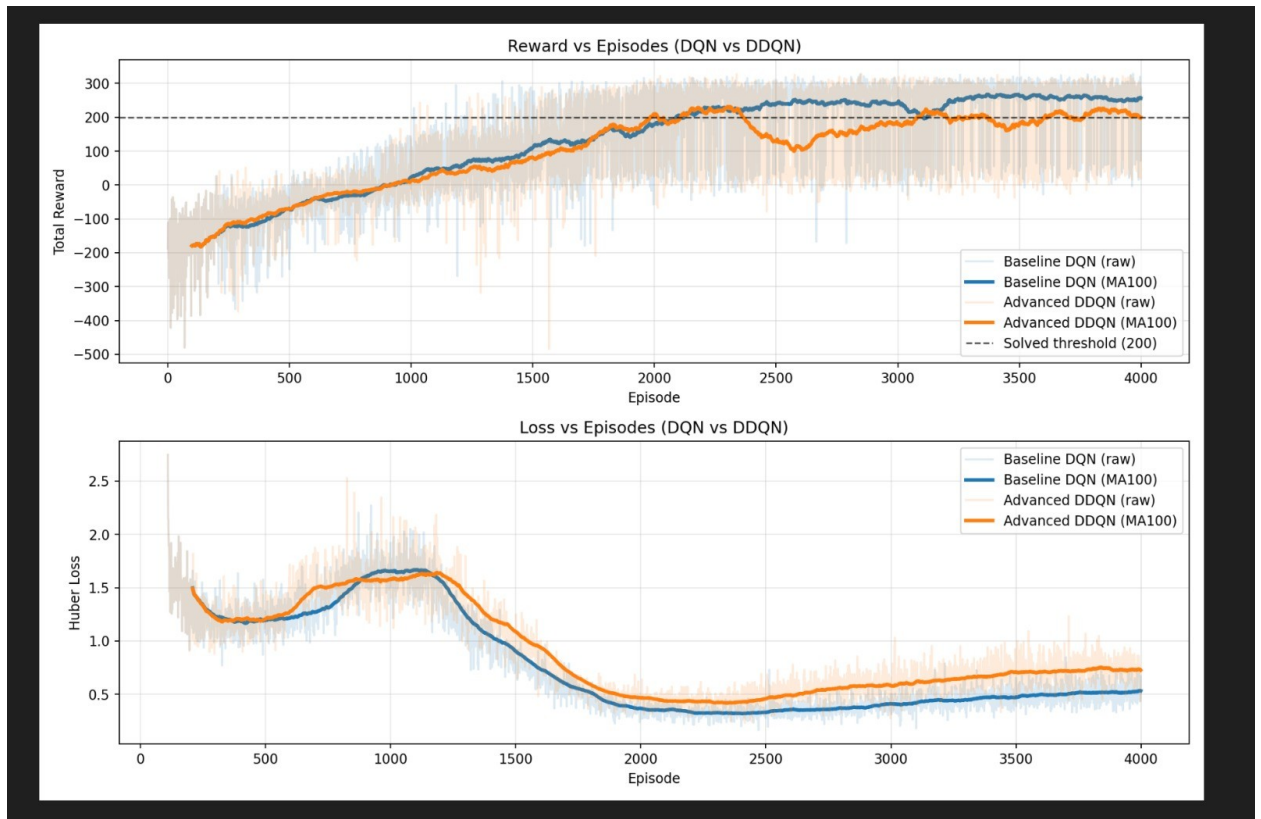
Hyperparameter	Value
Discount factor (γ)	0.99
Learning rate	$5e-4$
Batch size	128
Replay buffer capacity	300,000

Learning starts after	10,000 steps
Train frequency	Every 4 steps
Target update	Every 5,000 steps
Gradient clipping	10.0
ϵ start / decay / min	1.0 / 0.999 / 0.05
Smoothing window	100 episodes (MA100)
Evaluation episodes	100 (greedy)

5.2 Experiments

Experiment 1 (Required): Baseline DQN vs Advanced DDQN under identical architecture and hyperparameters. We present combined reward and loss curves for direct comparison.

Figure 2: Reward and Loss vs Episodes (DQN vs DDQN). Raw curves are shown lightly; MA100 is shown as bold lines.



We also monitor success rate and fuel remaining on successful landings to interpret control quality and resource usage.

Figure 3: Success Rate (MA100) — DQN (left) vs DDQN (right).

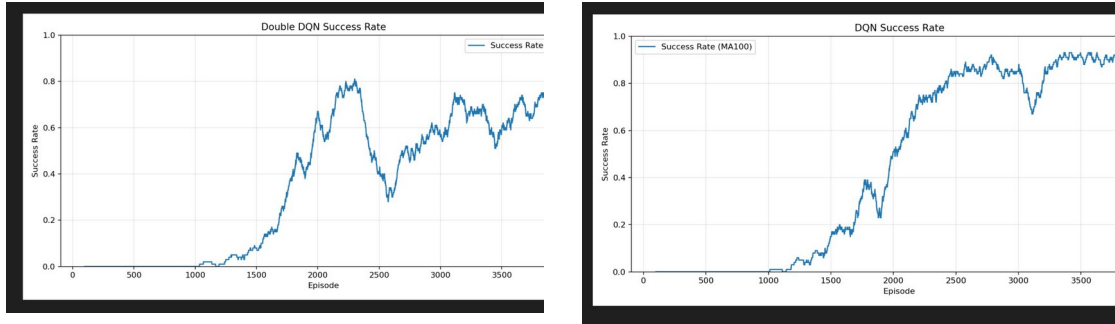
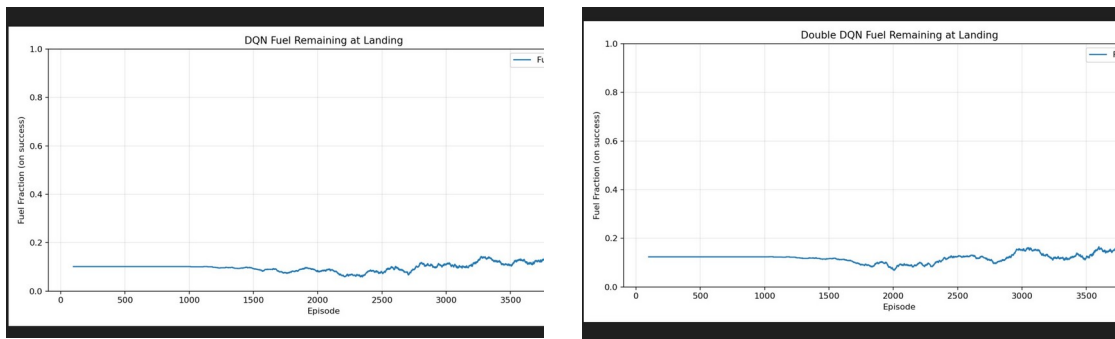


Figure 4: Fuel Remaining on Success (MA100) — DQN (left) vs DDQN (right).



5.3 Results Discussion (Deep Analysis)

Learning speed: Both agents improve from negative reward to consistently positive reward. The Baseline DQN MA100 curve increases more smoothly and ends higher. DDQN improves strongly early but shows a mid-training drop followed by recovery, indicating higher sensitivity/instability in this run.

Loss convergence: Both agents' smoothed Huber loss decreases as training progresses. DQN settles to a lower loss late in training than DDQN, which matches the more stable reward behavior.

Stability and success rate: DQN reaches a high final success rate with relatively small late-stage fluctuations. DDQN shows larger success-rate oscillations (peak, drop, then recovery). This suggests that in this specific run, DQN was more stable.

Fuel efficiency: Both agents show gradual improvement in fuel remaining on successful landings toward the end of training. DDQN ends slightly higher in fuel remaining, but this should be interpreted together with reward and success rate.

Theory-based explanation: DDQN reduces Q-value overestimation by decoupling selection and evaluation, but training dynamics depend on hyperparameters, seed, and training horizon. A robust conclusion should be supported by multi-seed evaluation and reporting mean $\pm$ std across seeds.

Table 1: Training-end summary (Episode 4000, MA100) — values read from the final smoothed training curves in Figures 2–4.

Agent	Final MA100 Reward	Stability (training)	Final MA100 Success (%)
DQN (Baseline)	≈ 265	More stable	≈ 89
DDQN (Advanced)	≈ 215	More fluctuation	≈ 65

6. Conclusion & Future Work

We implemented Baseline DQN and Advanced DDQN for LunarLander-v3 with a 9D state including fuel_fraction. Using MA100 smoothing and Matplotlib figures, we compared learning speed, loss convergence, and stability. In our run, DQN achieved higher final smoothed reward and a more stable success rate, while DDQN exhibited larger mid-training fluctuations.

Future Work: run multiple seeds and report mean $\pm$ std across seeds; test Prioritized Experience Replay, Dueling networks, and controlled fuel shaping.

7. References

Mnih, V. et al. (2015). Human-level control through deep reinforcement learning. Nature. <https://www.nature.com/articles/nature14236>

van Hasselt, H., Guez, A., & Silver, D. (2015). Deep Reinforcement Learning with Double Q-learning. arXiv:1509.06461. <https://arxiv.org/abs/1509.06461>

Gymnasium LunarLander documentation. https://gymnasium.farama.org/environments/box2d/lunar_lander/

Gymnasium Box2D environments and installation notes. <https://gymnasium.farama.org/environments/box2d/>

PyTorch documentation. <https://pytorch.org/>

Matplotlib documentation. <https://matplotlib.org/stable/>

NumPy documentation. <https://numpy.org/doc/>